Chapter - 3

TOKENS EXPRESSIONS AND CONTROL STRUCTURES

C++ Tokens

- The smallest individual unit of a program is known as token.
- C++ has the following tokens:
  - Keywords
  - Identifiers
  - Constants
  - Strings
  - Special Symbols
  - Operators

```
#include <iostream>
 using namespace std;
 int main()
 {
        cout << "Hello World";
        return 0;
 }
```

Keywords

- Explicitly reserved words, used to implement specific C++ language features.
- Each keyword has a special meaning.
- Cannot be used as names of program variables or other user defined program elements.
- i.e. We cannot use keywords as user-defined identifiers.
- A list of ANSI C and ANSI C++ keywords are as shown in the following table.

Keywords in C++

Table 3.1 *C++ keywords*

asm	**double**	new	**switch**
auto	**else**	operator	template
break	**enum**	private	this
case	**extern**	protected	throw
catch	**float**	public	try
char	**for**	**register**	**typedef**
class	friend	**return**	**union**
const	**goto**	**short**	**unsigned**
continue	**if**	**signed**	virtual
default	inline	**sizeof**	**void**
delete	**int**	**static**	**volatile**
do	**long**	**struct**	**while**
Added by ANSI C++			
bool	export	reinterpret_cast	typename
const_cast	false	static_cast	using
dynamic_cast	mutable	true	wchar_t
explicit	namespace	typeid	

Note: The ANSI C keywords are shown in bold face.

Identifiers

- Identifiers refer to names that are given to various user defined program elements, such as
  - Variables
  - Functions
  - Arrays
  - classes etc.
- They are fundamental requirement of any language.
- Each language has its own rules for naming an identifier

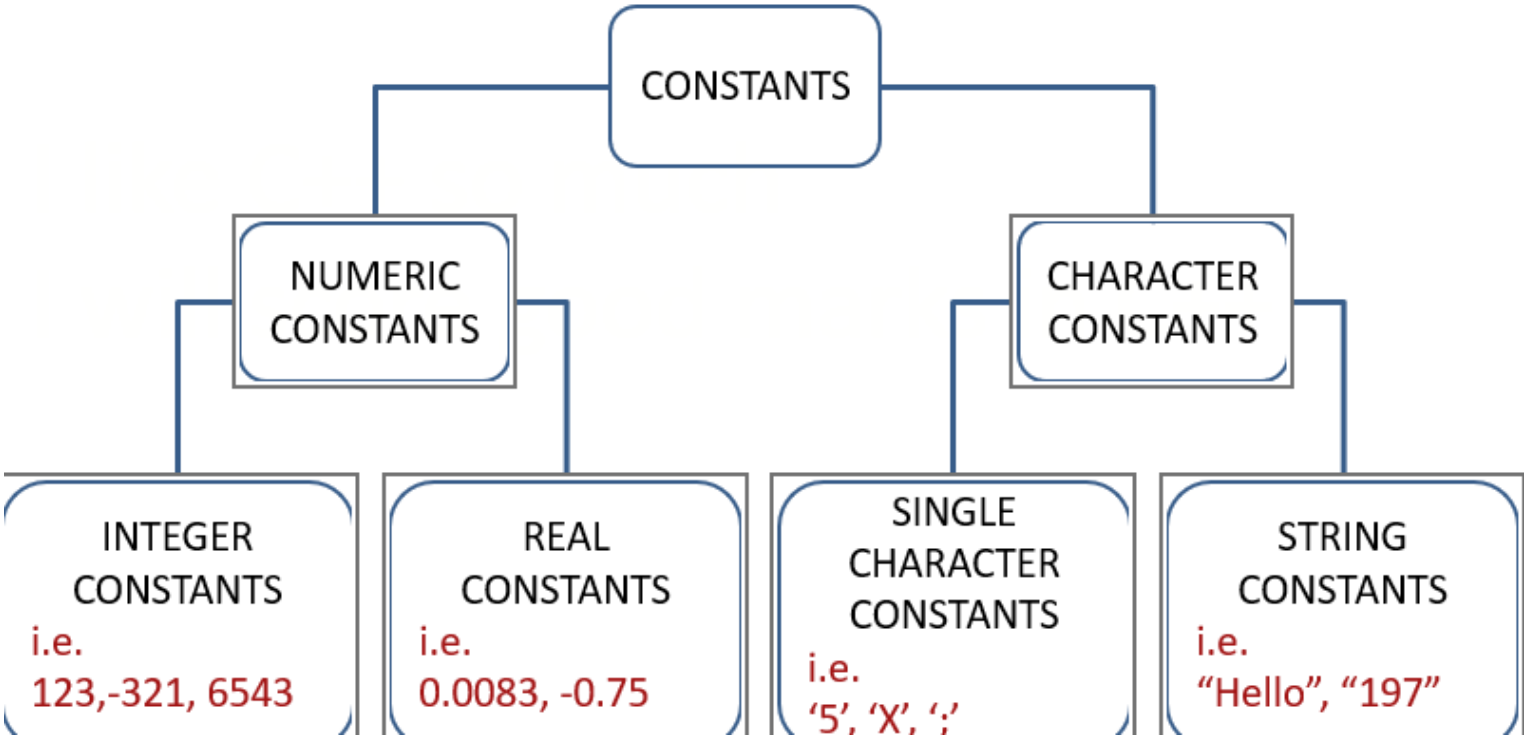
Rules for naming identifiers in C++

- Only alphabetic characters, digits and underscore are permitted
- First Character must be an alphabet or an underscore. The name cannot start with a digit.
- Uppercase and Lowercase letters are distinct. (Case Sensitive)
- A declared keyword cannot be used as a variable name.
- ANSI C recognizes only first 32 characters in a name, whereas ANSI C++ places no limit on the length.

Constants / Literals

- Constants in C++ refer to fixed values that do not change during execution of program.
- **Like C, C++ supports several kinds of Literal Constants**
  - INTEGERS
  - CHARACTERS
  - FLOATING POINT NUMBERS
  - STRINGS
- Literal constants do not have memory locations.

CONSTANTS


CONSTANTS
NUMERIC CONSTANTS
CHARACTER CONSTANTS
INTEGER CONSTANTS
i.e.
123,-321, 6543
REAL CONSTANTS
i.e.
0.0083, -0.75
SINGLE CHARACTER CONSTANTS
i.e.
'5', 'X', ';'
STRING CONSTANTS
i.e.
"Hello", "197"

Constants

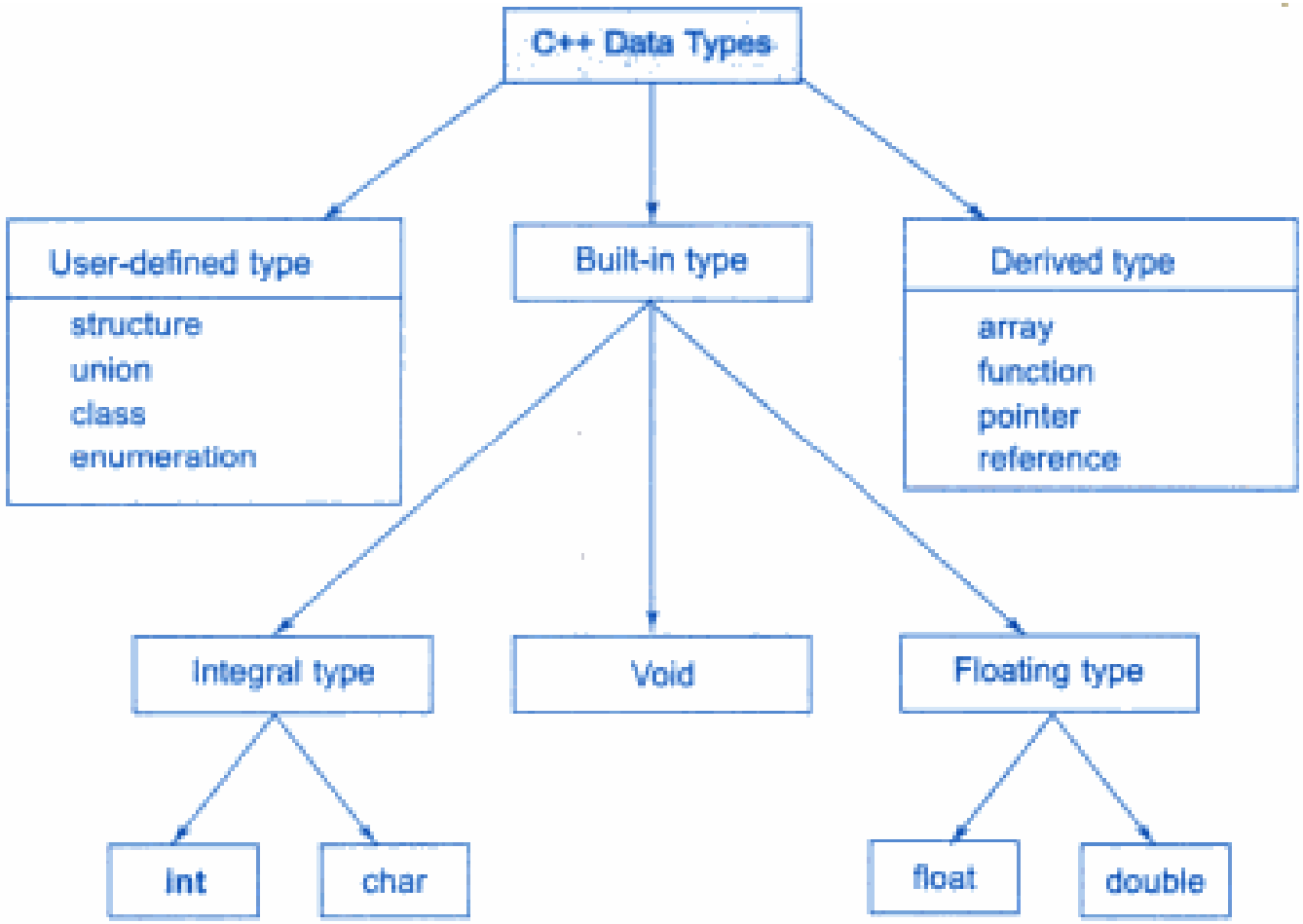
```
123             // decimal integer
12.34           // floating point integer
037             // octal integer
0X2             // hexadecimal integer
"C++"           // string constant
'A'             // character constant
L'ab'           // wide-character constant
```

- wchar_t is a wide character literal introduced by ANSI C++. It is for character sets that do not fit in a single byte.
- C++ also recognizes all backslash character constants available in C.
- C++ also recognizes all the backslash character constants available in C

Constants (Contd..)

- C++ supports 2 types of string representation
  - C Style character string
  - String Class type introduced by Standard C++

C++ Data Types


C++ Data Types
User-defined type
structure
union
class
enumeration
Built-in type
Derived type
array
function
pointer
reference
Integral type
Void
Floating type
int
char
float
double

Basic Data Types (16-bit machine)

Table 3.2 *Size and range of C++ basic data types*

Type	Bytes	Range
char	1	–128 to 127
unsigned char	1	0 to 255
signed char	1	– 128 to 127
int	2	– 32768 to 32767
unsigned int	2	0 to 65535
signed int	2	– 31768 to 32767
short int	2	– 31768 to 32767
unsigned short int	2	0 to 65535
signed short int	2	–32768 to 32767
long int	4	–2147483648 to 2147483647
signed long int	4	–2147483648 to 2147483647
unsigned long int	4	0 to 4294967295
float	4	3.4E–38 to 3.4E+38
double	8	1.7E–308 to 1.7E+308
long double	10	3.4E–4932 to 1.1E+4932

C++ has added two more datatypes : bool and wchar_t

Refer Program 3_1.cpp

Basic Data Types (cont.)

void Data type

It has three main uses

1. To specify the return type of a function when the function is not returning any value
2. To indicate an empty argument list of a function.

```
void function1(void);
```

3. To Declare Generic Pointer

```
void *gp;
```

Basic Data Types (cont.)

GENERIC POINTER

A generic pointer can be assigned a pointer value of any basic data type, but it may not be dereferenced.

Eg . int *ip // int pointer

gp = ip // Assign integer pointer to void pointer

*ip = *gp is illegal statement

You cannot dereference a pointer to a void value.

Basic Data Types (cont.)

- Assigning any pointer type to a void pointer without using a cast is allowed both in C and C++

  Like we did gp = ip; in previous slide.

- Assigning void pointer to non void pointer without cast is allowed in C but not in C++

```
void * p1;

char *p2;

p2 = p1;
```

C

```
p2 = (char *) p1;
```

C++

User Defined Data types

1. struct
2. union
3. class
4. enum

Enumeration

- It provides a way of attaching names to numbers
- By default, enumerators are assigned integer values starting with 0.
- It provides an alternative means to create symbolic constants
- The syntax of enum statement is similar to struct statement

  Eg : enum shape{circle, square, triangle};

  enum colour{red, blue, green, yellow};

- In C++ the tag names become new type names
- By using these tag names, we can declare new variables.

shape ellipse;

colour background;

Enumeration

- ANCI C defines the types of enums to be ints
- In C++, each enumerated datatype retains its own separate datatype.
- i.e C++ doesn't permit an int value to be automatically converted to enum value;
- Eg colour background = blue; // allowed

  colour background = 7; // not allowed (Error in C++)

  colour background = (colour) 7 ; //OK

However an enumerated value can be used in place of int value.

int c = red; //valid, colour type promoted to int.

Overriding values of enumerators

- By default enumerators are assigned integer values 0, 1,2, …..
- enum coin { penny, nickel, dime, quarter=100, half_dollar, dollar};
- The values of these symbols are

penny 0

nickel 1

dime 2

quarter 100

half_dollar 101

dollar 102

Annonymous enum

- enums without tag names
- enum { off, on };
- Here off is 0 and on is 1
- int switch1 = off;
- Int switch2 = on;

Enumeration as Symbolic Constants

Enumerators can be used as symbolic constants for switch statement

Refer Program 3_enum.cpp

Derived Data types

- Arrays
- Functions
- Pointers

Arrays

Application similar to C

Only Exception – Way they are initialized

In C – Compiler allows us to declare array size as the exact length of string constant. It assumes that the programmer intends to leave out the null character \0

- char string[3] = “xyz”;

In C_++ – The size should be one larger than the number of characters in the string.

- char string[4] = “xyz”;

Functions

Major changes in C++ compared to C

Explained in detail in Chapter No. 4

Pointers

- Extensively used for memory management
- Declared and initialized as in C

```
int * ip; // integer pointer declared
ip = &x; //address of x assigned to ip
*ip = 10; //Value 10 assigned to x
```

- C++ adds the concept of constant pointer and pointer to constant

```
char * const ptr1 = “GOOD”;
```

We cannot modify the address that ptr1 is initialized to.

Pointers (contd..)

- Ptr2 is declared as pointer to constant

int const *ptr2 = &m;

It can point to any variable of correct type, but the contents of what it points to cannot change.

- We can also declare both pointer and the variable as constant in the following way :

const char * const cp = “xyz”

Here neither the address assigned to pointer, nor the contents it points to can be changed.

Pointer to Constant(Eg.)

```
int a = 10, b = 20;

const int* ptr = &a;


*ptr = 15;     // ❌ Error: cannot modify value through pointer

ptr = &b;      // ☑ Allowed: can change what ptr points to
```

Here Data is constant, pointer is not constant.

Constant Pointer (Eg.)

```
int a = 10, b = 20;

int* const ptr = &a;


*ptr = 15;     // ☑ Allowed: can change value

ptr = &b;      // ❌ Error: cannot change pointer
```

Here Pointer is constant, but data is not constant

Constant Pointer to Constant (Eg.)

You cannot change the pointer, and you cannot change the value it points to.

```
int a = 10, b = 20;

const int* const ptr = &a;


*ptr = 15;     // ❌ Error: cannot change value

ptr = &b;      // ❌ Error: cannot change pointer
```

Declaration	Can modify value?	Can change pointer?
const int* ptr	✗ No	☑ Yes
int* const ptr	☑ Yes	✗ No
const int* const ptr	✗ No	✗ No

Symbolic Constants

- There are two ways of creating symbolic constants in c++.
  1. using the qualifier const.
  2. defining a set of integer constants using enum keywords.

In both C and C++, any value declared as const can't be modified by the program in any way.

- In C++, we can use const in a constant expression. Such as
  const int size = 10 ;
  char name (size) ;
  This would be illegal in C.

  - C++ requires a const to be initialized. ANSI C does not require an initializer; if none is given, it initializes the const to 0.
  - In C++ const values are local and in ANSI C const values are global. However they can be made local by declaring them as static.

- In C++ if we want to make const value as global then declare as extern storage class.

Eg: external const total=100;

Another method of naming integer constants is as follows:-

enum {x,y,z};

Type Compatibility

C++ is very strict with regards to type compatibility

C++ defines int, short int and long as 3 different types

Casting must be done when assigning values to one another.

Similarly for unsigned char, char and signed char

In C char is promoted to size of int, but in C++ it is not.

sizeof('x')equals to sizeof(char) in C++

sizeof('x')equals to sizeof(int) in C

Declaration of Variables

- Like in C, in C++ also all the variables must be declared before they are used in executable statements.
- Difference in place declaration of variables in C and C++
- C requires all variables to be declared at the beginning of scope
- C++ allows the declaration of variables anywhere in the scope.

Dynamic Initialization of variables

- C++ permits initialization of variables at runtime. This is referred to as dynamic initialization
- This can be done using expressions at the place of declaration

Eg : float area = 3.14 * radius * radius;

- Dynamic initialization is extensively used in OOP

Reference Variable

- A reference provides an alias or a different name for a variable.
- The two names can be used interchangeably to represent that variable

```
data-type & reference-name = variable-name
```

Example:

```
float total = 100;
float & sum = total;
```

- One of the most important uses for references is in passing arguments to functions

Both these statements will print 100

```
cout << total;
```

and

```
cout << sum;
```

- It is necessary to initialize the

reference variable at the time of declaration

```
int a = 10;

int &ref = a;  // ref is a reference to a
```

- ref is not a new variable. It just becomes another name for a.
- Any change to ref also changes a, and vice versa.

Key Points about Reference Variables:

Feature	Description
Must be initialized	A reference must be initialized at the time of declaration.
Cannot be NULL	References always refer to some valid object.
Cannot change reference	Once set to a variable, a reference cannot be made to refer to another.
No extra memory	It uses the same memory as the original variable.

Reference Variables (Cont.)

C++ assigns additional meaning to the symbol &

Here & is not address operator.

The notation float & means reference to float.

Some more examples :

```
int n[10];
int & x = n[10];
char & a = '\n';
```

Here x is alias to array n[10]
Initializing reference to a literal

```
int x;
int *p = &x;
int & m = *p;
```

Here m refers to x which is pointed by pointer p

```
int & n = 50;
```

Creates and int object with value 50 and name n

Major application of reference variable is passing arguments to function

```
void f(int & x)          // uses reference
{
   x = x+10;             // x is incremented; so also m
}
int main()
{
   int m = 10;
   f(m);                 // function call
   .....
   .....
}
```

C++ Operators

All C language operators are valid in C++.

1. Arithmetic operators (+, - , *, /, %)
2. Relational operators (<, <=, >, >=, ==, !=)
3. Logical operators (&&, ||, !)
4. Assignment operators (+=, -=, *=, /=)
5. Increment and decrement operators (++, --)
6. Conditional operators (?:)
7. Bitwise operators (&, |, ^, <<, >>)
8. Special operators ()

New Operators in C++

<<	Insertion operator
>>	Extraction Operator
::	Scope resolution operator It allows to access to the global version of variable
:: *	Declares a pointer to a member of a class
→*	To access pointer to class members
. *	To access pointer to data members of class
delete	Deallocates memory at run time
endl	It is a manipulator causes a linefeed to be inserted
new	Allocates memory at run time
setw	It is a manipulator specifies a field width for printing value

Scope Resolution Operator

The **scope resolution operator ::** is used in C++ to **access something that is out of the current scope**—such as a global variable, class member, or namespace item.

```
:: variable-name
```

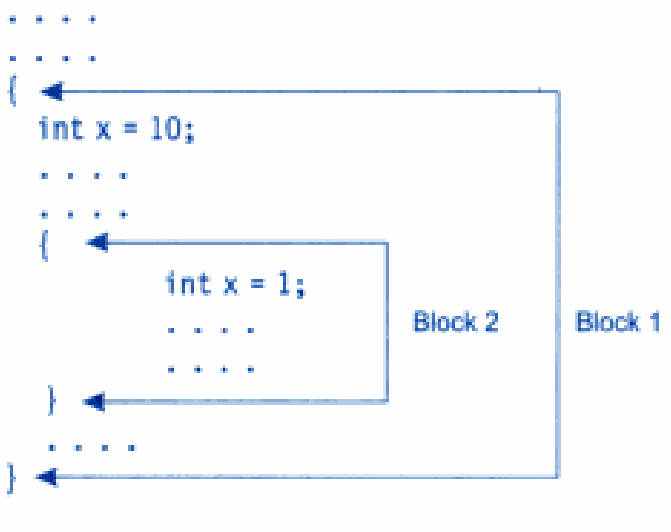
Scope Resolution Operator

When a **local variable hides a global variable** (i.e., both have the same name), we use :: to refer to the **global** one.

Global version of a variable in C++ can be accessed by using scope resolution operator.

```
:: variable-name
```

Scope Resolution Operator



Declaration of x in inner-block hides the declaration of x in outer block.

Global version of a variable in C++ can be accessed by using scope resolution operator.

```
#include <iostream>
using namespace std;

int x = 100;  // Global variable

int main() {
   int x = 50;  // Local variable
   cout << "Local x = " << x << endl;
   cout << "Global x = " << ::x << endl;  // Using scope resolution operator
   return 0;
}
```

The :: operator links the **member function definition** to the class.

```
class Demo
 {
public:
    void show();  // Declaration
};

// Definition outside the class using ::

void Demo :: show() {
    cout << "Hello from Demo class!";
}
```

Use of Scope Resolution Operator

Usage	Example	Purpose
Global variable access	::x	Access global variable shadowed by local
Class function definition	ClassName::funcName()	Define member function outside class
Namespace access	std::cout	Use standard namespace
Static member access	ClassName::staticVar	Define/access static data member

Member Dereferencing Operators

Table 3.3 *Member dereferencing operators*

Operator	*Function*
::*	To declare a pointer to a member of a class
*	To access a member using object name and a pointer to that member
–>*	To access a member using a pointer to the object and a pointer to that member

1. ::* – Pointer to Member Declaration

Used to **declare a pointer to a member** (not an object).

```
class MyClass {
public:
    int a;
};

int MyClass::*ptr = &MyClass::a;  // pointer to member 'a' of MyClass
```

2. * – Access Using Object Name and Member Pointer

Used when you have an **object** and a **pointer to member**, and want to access that member.

```
MyClass obj;
 obj.a = 10;
```

```
int MyClass::*ptr = &MyClass::a;
```

```
cout << obj.*ptr; // Outputs: 10
```

3. ->* – Access Using Object Pointer and Member Pointer

Used when you have a **pointer to object** and a **pointer to member**.

```
MyClass obj;
MyClass* pObj = &obj;
obj.a = 20;

int MyClass::*ptr = &MyClass::a;
cout << pObj->*ptr;  // Outputs: 20
```

Memory Management Operators

C uses malloc() and calloc() functions to allocate memory dynamically at run time.

Similarly it uses free() to free dynamically allocated memory

C++ supports these functions and additionally defines two unary operators new and delete. They are also known as free store operators.

New Operator

```
pointer-variable  =  new data-type;
```

```
int *p = new int;
float *q = new float;
```

We can also initialize the memory using the new operator

```
pointer-variable  =  new data-type(value);
```

```
int *p = new int(25);
float *q = new float(7.5);
```

new can be used to create memory space for arrays also

```
pointer-variable  =  new data-type[size];
```

```
int *p = new int[10];
```

When creating multi dimentsional arrays with new, all array sizes must be supplied.

delete Operator

When a data object is no longer needed, it is destroyed to release memory space for reuse.

The general form of its use is

```
delete pointer-variable;
```

If we want to free a dynamically allocated array, we must use the following form of delete

```
delete [size] pointer-variable;
```

Recent versions of C++ do not require to specify size

delete [ ]p; will delete entire array pointed by p.

The new operator offers the following advantages over the function malloc()

- It automatically computes the size of the data object. We need not use the operator sizeof.
- It automatically returns the correct pointer type, so that there is no need to use a type cast.
- It is possible to initialize the object while creating the memory space.
- Like any other operator, new and delete can be overloaded.

Manipulators

- Manipulators are operator that are used to format the data display. The most commonly manipulators are **endl and setw.**
- The endl manipulator, when used in an output statement, causes a line feed to be insert.(just like \n)

```
cout << “n=“ << n << endl;
```

- If we want to print all numbers in right justified way use setw() which specify a common field width

```
cout<< setw(5) << sum << endl;
```

Type Conversion

Type Conversion is the process of converting one predefined data type into another data type.

Explicit type conversion is also known as type casting.

Type Casting

In C++ explicit type conversion is called type casting.

Syntax

type-name (expression) //C++ notation

Example

```
average = sum/(float) i; //C notation
average = sum/float (i); //C++ notation
```

Expressions and their types

Constant expressions
Integral expressions
Float expressions
Pointer expressions
Relational expressions
Logical expressions
Bitwise expressions

Special Assignment Expressions

- Chained Assignment

```
x = (y = 10);
    or
x = y = 10;
```

- Embedded Assignment

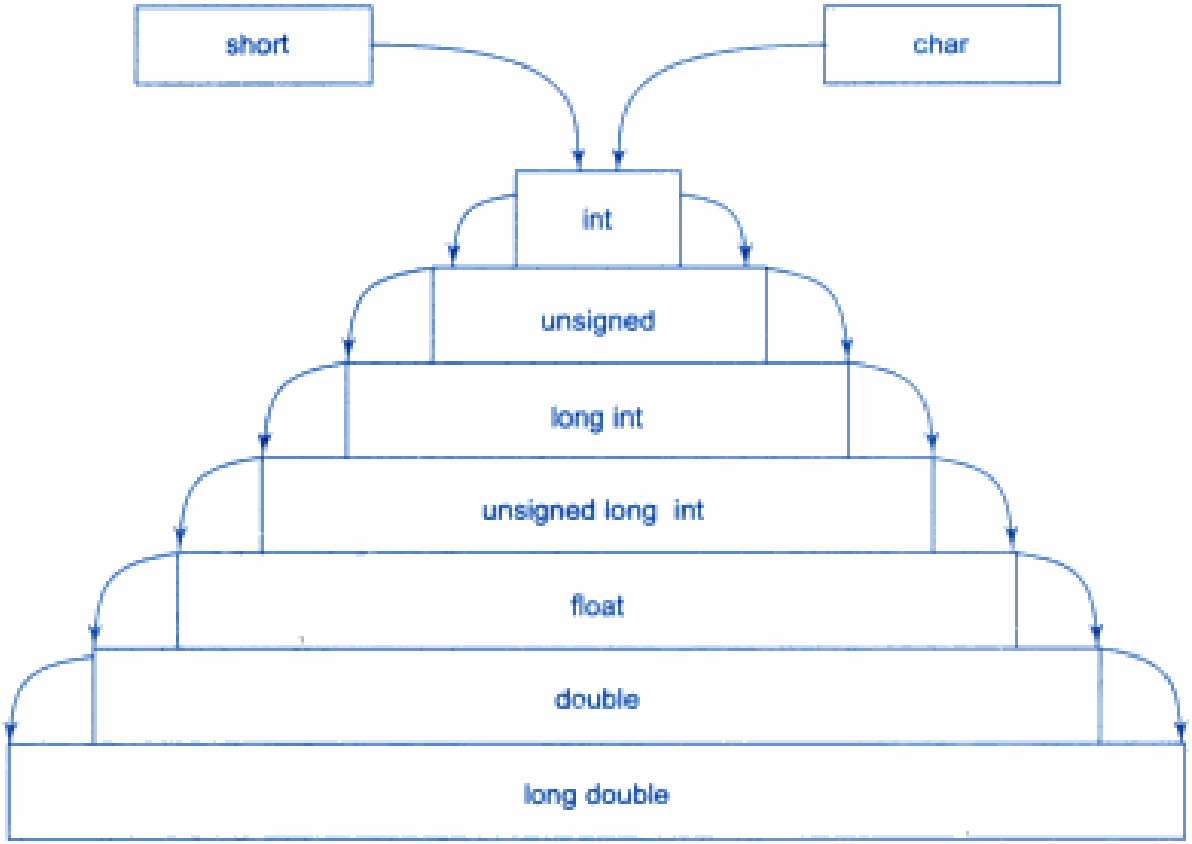
```
x = (y = 50) + 10;
```

- Compound Assignment

```
variable1 op= variable2;
x += 10;
```

Implicit Conversions

Smaller data type is converted to a wider data type



Implicit conversions

Table 3.4 *Results of Mixed-mode Operations*

LHO \ RHO	*char*	*short*	*int*	*long*	*float*	*double*	*long double*
char	int	int	int	long	float	double	long double
short	int	int	int	long	float	double	long double
int	int	int	int	long	float	double	long double
long	long	long	long	long	float	double	long double
float	float	float	float	float	float	double	long double
double	double	double	double	double	double	double	long double
long double	long double	long double	long double	long double	long double	long double	long double

RHO – Right-hand operand
LHO – Left-hand operand

Operator Overloading

- Defining additional meaning to operators
- << Left shift as well as insertion operator
- + operator can be overloaded to add two integer values and concate two string values.
- Operator overloading is explained in detail in Chapter 7

Operator Precedence

Operator	*Associativity*
::	left to right
–> . ( ) [ ] postfix ++ postfix – –	left to right
prefix ++ prefix – – ~ ! unary + unary – unary * unary & (type) sizeof new delete	right to left
–> * *	left to right
* / %	left to right
+ –	left to right
<< >>	left to right
<< = >> =	left to right
= = !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
?:	left to right
= * = / = % = + = =	right to left
<< = >> = & = ^= |=	left to right
, (comma)	

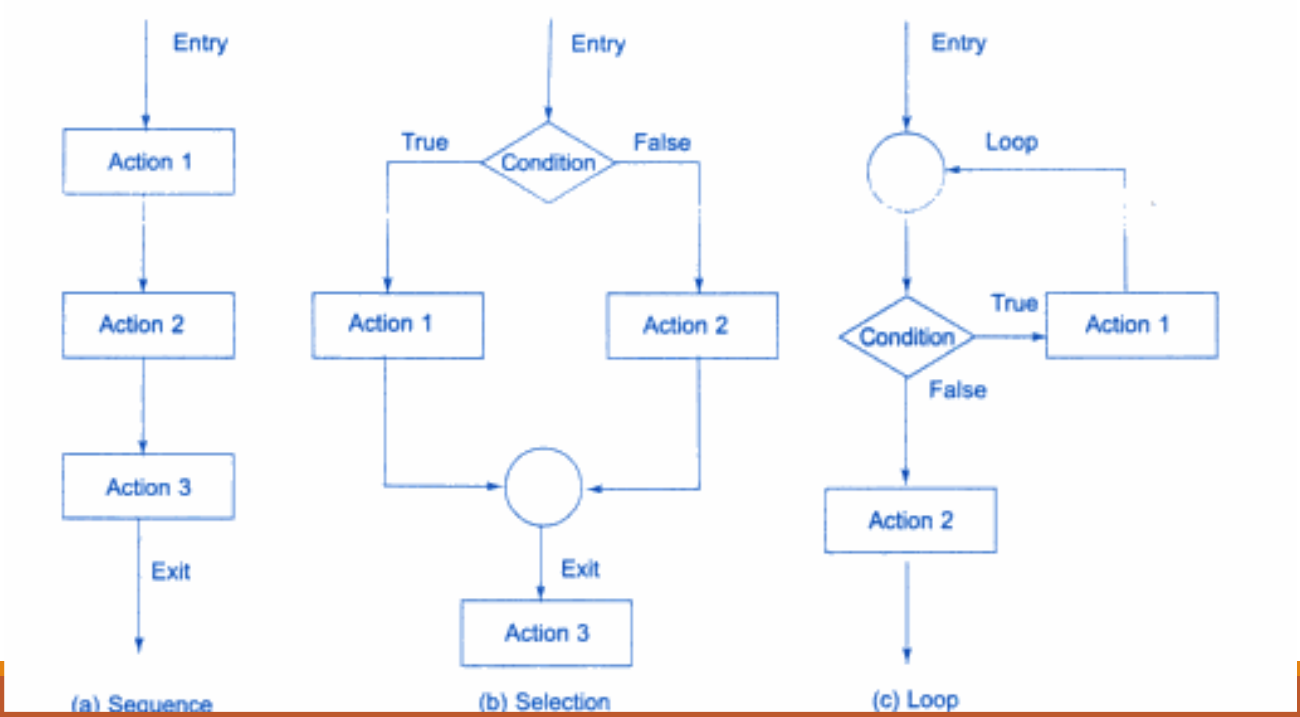
The unary operations assume higher precedence.

Control Structures

Sequence Structure (Straight Line)

Selection Structure (Branching)

Loop Structure (Iteration or repetation)



Control Structures

- The if statement:
  - Simple if statement
  - if…else statement
  - else…if ladder
  - if…else nested
- The switch statement
- The do-while statement: An exit controlled loop
- The while Statement: An entry controlled loop
- The for statement: An entry controlled loop

Selection Statements (Syntax)

```
if(expression is true)
{
    action1;
}
action2;
action3;
```

```
if(expression is true)
{
    action1;
}
else
{
    action2;
}
action3;
```

Syntax of Switch Statement

```
switch(expression)
{
   case1:
   {
   action1;
   }
   case2:
   {
   action2;
   }
   case3:
   {
   action3;
   }
   default:
   {
   action4;
   }
}
action5;
```

Loops (Syntax)

```
do
{
   action1;
}
while(condition is true);
action2;
```

```
while(condition is true)
{
   action1;
}
action2;
```

```
for(initial value; test; increment)
{
   action1;
}
action2;
```